

or counting type (or a combination of the two), depending on the requirements.

In our previous article, “The Basics of the System V Semaphore”, *LFY, July 2006*, page 86-89, we explained the binary semaphore’s primitives; this article will explain the intricacies of complex semaphore programs.

Different functions are required to create and access a semaphore. To create a semaphore set, the *semget* function is used:

```
int semget ( key_t key , int Number of Semaphores, int SemaphoreFlag);
```

The first argument *key* is the unique identifier for an IPC resource. The second argument is the number of semaphores in the semaphore set, and the third argument is the semaphore flag. The flag *IPC_CREAT* is used to create a new semaphore set. When *IPC_CREAT|IPC_EXCL* is used, the system call returns -1 if the semaphore set already exists. Otherwise, the call creates a new semaphore set and returns zero.

semctl() is a system call used to perform control operations on the semaphore set. The syntax of the call is as follows:

```
int semctl( int SemaphoreID , int SemaphoreNumber, int Command, union semun);
```

...where the structure of *union semun* is:

```
union semun{
    int val;
    struct semid_ds *buffer;
    unsigned short * array;
}
```

The first member of the *union semun*, *val*, sets the value of a semaphore. Here is where programmers decide whether they need the binary or counting semaphore. If the *val* is assigned to 1, then it becomes a binary semaphore and is used to perform mutual exclusion; if the value is greater than 1 then the semaphore is of the counting type and the initial value of the first member is set to be equivalent to the number of shared resources. The second member, the buffer, is a data structure describing a set of semaphores. The next member is the array, which is used to set or get the value for all the created semaphores by passing the *SETALL* or *GETALL* command correspondingly to *semctl* function.

The following code snippet [Program 1] explains how to create two semaphores in the semaphore set, in which one is a binary and the other a counting semaphore with an initial value of 2.

```
1 /* SemMulCr.c This program demonstrates
2  * creation of multiple semaphores */
3 #include <stdio.h>
4 #include <sys/sem.h>
5
```

```
6 union semun {
7     int val;
8     unsigned short int *array;
9 }arg;
10
11 main() {
12     int keysamid,ret;
13     static ushort semarray[2]={1,2};
14
15     key=ftok(".", "m");
16     semid=semget(key,2,IPC_CREAT |0744);
17     argarray=semarray;
18     semctl(semid,0,SETALL,arg);
19     for(i=0;i<2;i++){
20         ret=semctl(semid,i,GETVAL,0);
21         printf("Sem %d=%d\n",i,ret);
22     }
23 }
```

Notice that the semaphores have been created by passing the *SETALL* command as seen in Line 18 in the *semctl* function, and are retrieved through the *GETVAL* function in Line 20.

Now let us look at how to use these semaphores for synchronisation between two different processes. Locking or unlocking a critical section by means of decrementing or incrementing the semaphore’s count value is done by the *semop* system call. The increment and the decrement operations are executed atomically on selected members of the semaphore set indicated by the *SemaphoreID*. The second argument of the *semop* system call is a pointer to the array of semaphore operations structure *sembuf*, which has the following three members:

```
struct sembuf {
    short sem_num;
    short sem_op;
    short sem_flg;
};
```

A set of operations takes place, each being performed on every semaphore in the semaphore set, where the first member of the *sembuf* structure is the semaphore number (here it is the first semaphore of the set, so it is *Semaphore 0*); and the second member, *sem_op*, specifies whether the semaphore operation is incrementing or decrementing the value of the semaphore. The third member is the *sem_flg*, which is the operation flag that is used to specify whether the calling process can wait for the semaphore or not.

The third argument *Nbuffer* of the *semop* system call specifies the number of *sembuf* structures in the array.

A particular process can use a binary semaphore to access a critical section, but what if the process encounters more than one critical section? In that case, there should be more than one semaphore to synchronise multiple critical sections. Linux provides